

Lab 07 – Grid Layout, TextBoxes, Buttons, Events & Simple Validation

Title: Building Interactive Forms with Layouts and Event Handling

Objectives

The goal here was to create an interactive form where users could input employee data. I needed to use a more sophisticated layout (Grid) to arrange controls precisely, handle button click events, and implement client-side validation to prevent invalid data from being submitted. This is where the application starts to feel like a real tool rather than just a display.

Implementation

MainWindow.xaml

```
<Window x:Class="SalariiUI.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Sistem Salarii - Adăugare Angajat"
    Height="550"
    Width="650"
    WindowStartupLocation="CenterScreen">

    <Window.Resources>

        <SolidColorBrush x:Key="PrimaryBrush" Color="#2C3E50"/>
        <SolidColorBrush x:Key="AccentBrush" Color="#3498DB"/>
```

```
<SolidColorBrush x:Key="DangerBrush" Color="#E74C3C"/>
```

```
<SolidColorBrush x:Key="SuccessBrush" Color="#27AE60"/>
```

```
<Style TargetType="Label">
```

```
    <Setter Property="FontFamily" Value="Segoe UI"/>
```

```
    <Setter Property="FontSize" Value="14"/>
```

```
    <Setter Property="Foreground" Value="{StaticResource PrimaryBrush}"/>
```

```
    <Setter Property="Margin" Value="5,2"/>
```

```
    <Setter Property="FontWeight" Value="SemiBold"/>
```

```
    <Setter Property="VerticalAlignment" Value="Center"/>
```

```
</Style>
```

```
<Style TargetType="TextBox">
```

```
    <Setter Property="FontSize" Value="14"/>
```

```
    <Setter Property="Padding" Value="8,5"/>
```

```
    <Setter Property="Margin" Value="5,2"/>
```

```
    <Setter Property="BorderBrush" Value="#BDC3C7"/>
```

```
    <Setter Property="BorderThickness" Value="1"/>
```

```
    <Setter Property="Validation.ErrorTemplate">
```

```
        <Setter.Value>
```

```
            <ControlTemplate>
```

```
                <Border BorderBrush="Red" BorderThickness="2">
```

```
                    <AdornedElementPlaceholder/>
```

```
                </Border>
```

```
            </ControlTemplate>
```

```
        </Setter.Value>
```

```
    </Setter>
```

```
</Style>
```

```
<Style TargetType="Button">
```

```

        <Setter Property="FontSize" Value="14"/>
        <Setter Property="Padding" Value="15,8"/>
        <Setter Property="Margin" Value="5,2"/>
        <Setter Property="BorderThickness" Value="0"/>
        <Setter Property="Cursor" Value="Hand"/>
        <Setter Property="Foreground" Value="White"/>
    </Style>
</Window.Resources>

<Grid Margin="20">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/>
    </Grid.RowDefinitions>

    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="120"/>
        <ColumnDefinition Width="*/>
        <ColumnDefinition Width="Auto"/>
    </Grid.ColumnDefinitions>

    <!-- Title -->

```

```
<Border Grid.Row="0" Grid.ColumnSpan="3" Background="{StaticResource  
PrimaryBrush}"
```

```
Padding="15" Margin="0,0,0,15">
```

```
<TextBlock Text=" + Adăugare Angajat"
```

```
FontSize="20"
```

```
FontWeight="Bold"
```

```
Foreground="White"
```

```
HorizontalAlignment="Center"/>
```

```
</Border>
```

```
<!-- Nume -->
```

```
<Label Grid.Row="1" Grid.Column="0" Content="Nume:*/>
```

```
<TextBox Grid.Row="1" Grid.Column="1" x:Name="txtNume"  
TextChanged="ValidateForm"/>
```

```
<TextBlock Grid.Row="1" Grid.Column="2" x:Name="txtNumeError" Text="*"   
Foreground="Red"
```

```
Visibility="Collapsed" FontSize="16" FontWeight="Bold" Margin="5,0"/>
```

```
<!-- Prenume -->
```

```
<Label Grid.Row="2" Grid.Column="0" Content="Prenume:*/>
```

```
<TextBox Grid.Row="2" Grid.Column="1" x:Name="txtPrenume"  
TextChanged="ValidateForm"/>
```

```
<TextBlock Grid.Row="2" Grid.Column="2" x:Name="txtPrenumeError" Text="*"   
Foreground="Red"
```

```
Visibility="Collapsed" FontSize="16" FontWeight="Bold"/>
```

```
<!-- Funcție -->
```

```
<Label Grid.Row="3" Grid.Column="0" Content="Funcție:*/>
```

```
<TextBox Grid.Row="3" Grid.Column="1" x:Name="txtFuncție"  
TextChanged="ValidateForm"/>
```

```
<TextBlock Grid.Row="3" Grid.Column="2" x:Name="txtFuncțieError" Text="*"   
Foreground="Red"
```

Visibility="Collapsed" FontSize="16" FontWeight="Bold"/>

<!-- Departament -->

<Label Grid.Row="4" Grid.Column="0" Content="Departament:"/>

<ComboBox Grid.Row="4" Grid.Column="1" x:Name="cmbDepartament"

FontSize="14" Padding="8,5" Margin="5,2">

<ComboBoxItem>IT</ComboBoxItem>

<ComboBoxItem>Marketing</ComboBoxItem>

<ComboBoxItem>Finante</ComboBoxItem>

<ComboBoxItem>HR</ComboBoxItem>

<ComboBoxItem>Productie</ComboBoxItem>

<ComboBoxItem>Logistica</ComboBoxItem>

</ComboBox>

<!-- Salariu Brut -->

<Label Grid.Row="5" Grid.Column="0" Content="Salariu Brut:*/>

<TextBox Grid.Row="5" Grid.Column="1" x:Name="txtSalariuBrut"
TextChanged="ValidateForm"/>

<TextBlock Grid.Row="5" Grid.Column="2" x:Name="txtSalariuBrutError" Text="*"
Foreground="Red"

Visibility="Collapsed" FontSize="16" FontWeight="Bold"/>

<!-- Buttons -->

<StackPanel Grid.Row="6" Grid.Column="0" Grid.ColumnSpan="3"

Orientation="Horizontal" HorizontalAlignment="Center" Margin="0,10">

<Button x:Name="btnAdauga" Content="Adaugă Angajat"

Background="{StaticResource AccentBrush}"

Click="BtnAdauga_Click" IsEnabled="False"/>

<Button Content="Resetează"

Background="{StaticResource PrimaryBrush}"

Click="BtnReset_Click"/>

```

        <Button Content="Închide"
                Background="{StaticResource DangerBrush}"
                Click="BtnInchide_Click"/>
    </StackPanel>

    <!-- Status Message -->

    <Border Grid.Row="7" Grid.ColumnSpan="3" Background="{StaticResource LightBrush}"
            CornerRadius="5" Padding="10" Margin="0,10">
        <TextBlock x:Name="txtStatus" Text="Completați toate câmpurile obligatorii (*)"
                Foreground="{StaticResource PrimaryBrush}"
                FontSize="13" TextWrapping="Wrap"/>
    </Border>
</Grid>
</Window>

```

Code-Behind (MainWindow.xaml.cs)

```

using System;
using System.Windows;
using System.Windows.Controls;

namespace SalariiUI
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        private void ValidateForm(object sender, TextChangedEventArgs e)

```

```
{  
    bool isValid = true;  
  
    // Validate Nume  
    if (string.IsNullOrEmpty(txtNume.Text))  
    {  
        txtNumeError.Visibility = Visibility.Visible;  
        isValid = false;  
    }  
    else  
    {  
        txtNumeError.Visibility = Visibility.Collapsed;  
    }  
  
    // Validate Prenume  
    if (string.IsNullOrEmpty(txtPrenume.Text))  
    {  
        txtPrenumeError.Visibility = Visibility.Visible;  
        isValid = false;  
    }  
    else  
    {  
        txtPrenumeError.Visibility = Visibility.Collapsed;  
    }  
  
    // Validate Functie  
    if (string.IsNullOrEmpty(txtFunctie.Text))  
    {  
        txtFunctieError.Visibility = Visibility.Visible;  
        isValid = false;  
    }  
}
```

```

    }

    else

    {

        txtFuncțieError.Visibility = Visibility.Collapsed;

    }


    // Validate Salariu Brut (must be numeric and > 0)
    if (!double.TryParse(txtSalariuBrut.Text, out double salariu) || salariu <= 0)
    {

        txtSalariuBrutError.Visibility = Visibility.Visible;

        isValid = false;

    }

    else

    {

        txtSalariuBrutError.Visibility = Visibility.Collapsed;

    }


    btnAdauga.IsEnabled = isValid;

}


private void BtnAdauga_Click(object sender, RoutedEventArgs e)
{
    try
    {
        if (!btnAdauga.IsEnabled)
        {

            txtStatus.Text = "❌ Completați toate câmpurile obligatorii!";
            txtStatus.Foreground = System.Windows.Media.Brushes.Red;

            return;

        }
    }

```

```

// Parse the salary
double salariuBrut = double.Parse(txtSalariuBrut.Text);

// Create employee object
var angajat = new SalariiModel.Angajat
{
    Nume = txtNume.Text.Trim(),
    Prenume = txtPrenume.Text.Trim(),
    Functie = txtFunctie.Text.Trim(),
    Departament = (SalariiModel.Departament)cmbDepartament.SelectedIndex,
    SalariuBrut = salariuBrut,
    DataAngajarii = DateTime.Today
};

// Here you would save the employee to storage
// admin.AddAngajat(angajat);

txtStatus.Text = $"✅ Angajat {angajat.NumeComplet} adăugat cu succes!";
txtStatus.Foreground = System.Windows.Media.Brushes.Green;

// Reset form
ResetForm();
}
catch (Exception ex)
{
    txtStatus.Text = $"❌ Eroare: {ex.Message}";
    txtStatus.Foreground = System.Windows.Media.Brushes.Red;
}
}

```

```

private void BtnReset_Click(object sender, RoutedEventArgs e)
{
    ResetForm();
}

private void ResetForm()
{
    txtNume.Text = "";
    txtPrenume.Text = "";
    txtFunctie.Text = "";
    txtSalariuBrut.Text = "";
    cmbDepartament.SelectedIndex = 0;
    btnAdauga.IsEnabled = false;

    txtNumeError.Visibility = Visibility.Collapsed;
    txtPrenumeError.Visibility = Visibility.Collapsed;
    txtFunctieError.Visibility = Visibility.Collapsed;
    txtSalariuBrutError.Visibility = Visibility.Collapsed;

    txtStatus.Text = "Completați toate câmpurile obligatorii (*)";
    txtStatus.Foreground = System.Windows.Media.Brushes.Black;
}

private void BtnInchide_Click(object sender, RoutedEventArgs e)
{
    this.Close();
}
}

```

Why I Used This Approach

Grid Layout: I used a Grid with defined rows and columns for precise control over control placement. This is the standard layout for forms in WPF, as it allows consistent alignment of labels and input fields.

Validation with Visual Feedback: Each required field has an accompanying TextBlock with a red asterisk (*) that becomes visible when validation fails. This provides immediate visual feedback to the user about which fields need attention.

TextChanged Event: The `ValidateForm` method is called whenever text changes in any input field. This provides real-time validation feedback.

TryParse for Numeric Validation: Using `double.TryParse` ensures the salary field contains a valid number. This prevents crashes when the user enters non-numeric data.

IsEnabled Binding: The Add button is disabled until all fields pass validation. This prevents the user from submitting invalid data.

Error Handling: The try-catch block in `BtnAdauga_Click` handles any unexpected errors and displays them in the status message.

Reset Functionality: The Reset button clears all fields and error indicators, providing a clean slate for the user.

Status Message: A status message area provides feedback to the user about validation errors or successful operations.

Testing & Results

I tested the form by:

1. Entering valid data (all fields filled, salary as a number) – the Add button became enabled, and clicking it showed a success message
2. Leaving a required field empty – the red asterisk appeared for that field, and the Add button was disabled
3. Entering non-numeric data for salary – the salary error indicator appeared
4. Clicking Reset – all fields cleared and error indicators disappeared

The validation worked correctly, providing clear feedback about what was wrong with the input.

Reflection

This lab showed me how crucial validation is in UI applications. Users need clear feedback about what's wrong with their input. The pattern of showing error indicators inline with the fields (rather than just a generic error message) is much more user-friendly. I also learned about routing events – the Click event on the Button triggers the handler in the code-behind, which is the standard way to handle user actions in WPF. The real-time validation using TextChanged events provides a better user experience than validating only on submission.